

DDL & DML

Recap

- **Aggregation** to summarize many rows into one value
- **Grouping** to aggregate per group, not the whole table
- **Subquery** to filter one table using results from another
- **Join** to combine columns from multiple tables

Recap

`GROUP BY` to group rows by a column

`count(*)` to count the number of rows in each group

`Having` to filter groups based on the aggregated value

I know that she attended the SQL Symphony Concert 3 times in December 2017.

```
select person_id , count(*) as num_checkins
from facebook_event_checkin
where event_name like "%SQL Symphony Concert%"
and date between 20170101 and 20171231
group by person_id
having num_checkins = 3
```

Recap

Subquery to filter one table using results from another

```
select *  
from person  
where license_id in (...)  
and id in (...)
```

Join to combine columns from multiple tables

```
select *  
from person join drivers_license  
on person.license_id = drivers_license.id  
where drivers_license.hair_color = 'red';
```

Data Definition Language (DDL)

To define and modify the structure of a database

- **CREATE TABLE** : create a new table
- **ALTER TABLE** : modify a table
- **DROP TABLE** : remove a table

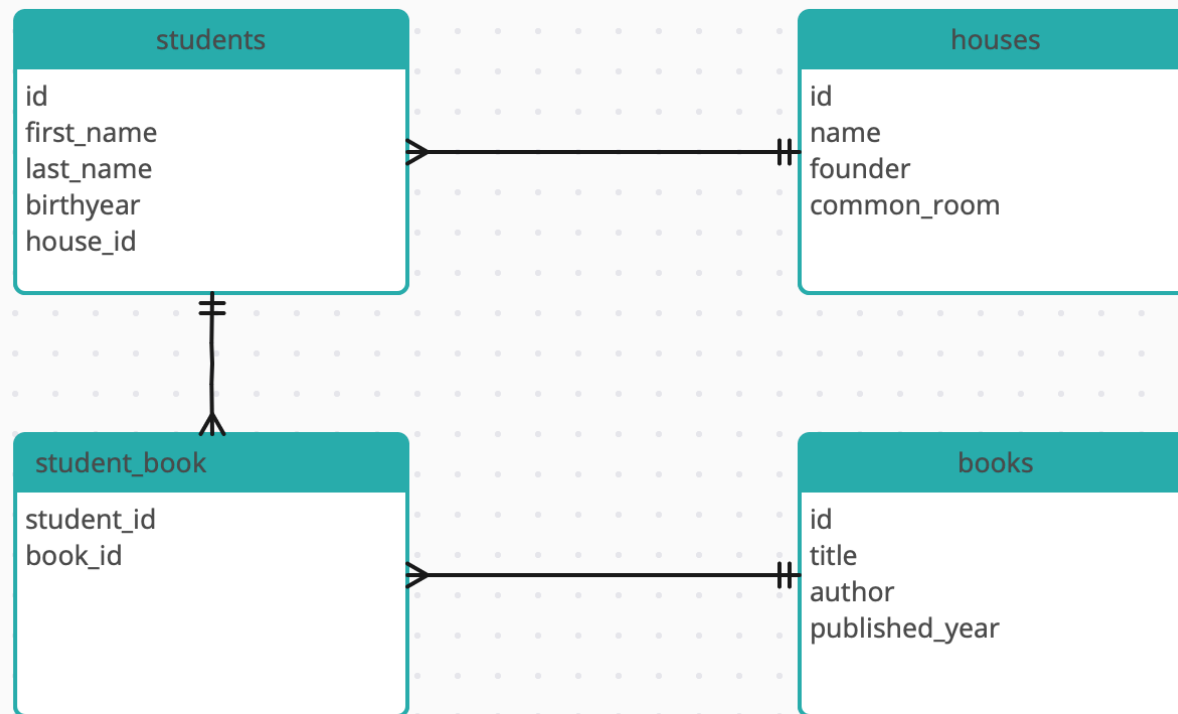
Data Manipulation Language (DML)

To manipulate the data in the database

- **INSERT INTO** : insert data into a table
- **UPDATE** : update data in a table
- **DELETE FROM** : remove data from a table

Complete the Harry Potter database

Two tables are missing: `books` and `student_book`.



CREATE TABLE to create a new table

```
-- syntax
CREATE TABLE table_name (
    column_name1 data_type, -- column definition (name and data type)
    column_name2 data_type, -- each column definition separated by a comma
    ...
    column_nameN data_type -- no comma after the last column definition
);
```

```
-- example
CREATE TABLE books (
    id numeric,
    title text,
    author text,
    published_year numeric
);
SELECT * FROM books; -- to confirm that the table is created; empty table
```

Data types in SQL

numeric

- int
- decimal
- float

text

- varchar(n)
- tinytext
- mediumtext

CREATE TABLE with constraints

```
CREATE TABLE books (  
    -- column definitions  
    id int,  
    title text,  
    author text,  
    published_year int,  
  
    -- table-level constraints defined at the end  
    PRIMARY KEY (id),           -- id is the primary key  
    CHECK (published_year >= 1900) -- published_year must be >= 1900  
);
```

Types of constraints

Constraints limit what values can be stored in a table

- `primary key` : `unique` & `not null`
 - `unique` : must be unique
 - `not null` : must have a value
- `foreign key` : must match a value in the referenced table
- `check (exp)` : must satisfy the expression
- ...

https://www.sqlite.org/lang_createtable.html

PRIMARY KEY and FOREIGN KEY constraints

PK: EMPLOYEES.employee_ID

➔ uniquely identifies each employee.

➔ who is this employee?

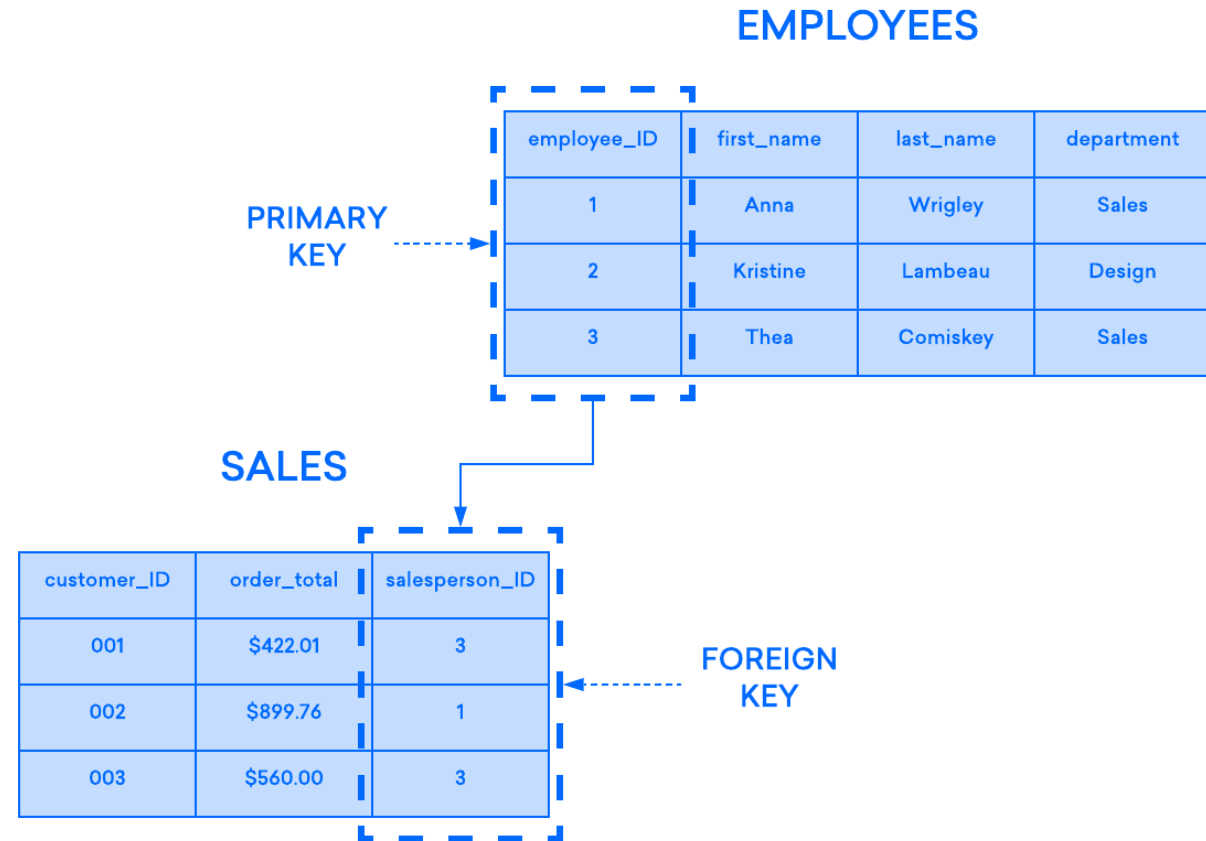
FK: SALES.salesperson_ID

➔ references EMPLOYEES.employee_ID

➔ who's responsible for the sale?

Referential integrity

➔ SALES.salesperson_ID must match an EMPLOYEES.employee_ID.



PRIMARY KEY and FOREIGN KEY constraints (continued)

```
CREATE TABLE EMPLOYEES (  
    employee_ID numeric,  
    first_name text,  
    last_name text,  
    department text,  
  
    -- employee_ID is the primary key  
    PRIMARY KEY (employee_ID)  
);  
  
CREATE TABLE SALES (  
    customer_ID numeric,  
    order_total numeric,  
    salesperson_ID numeric,  
  
    -- salesperson_ID is the foreign key → EMPLOYEES.employee_ID  
    FOREIGN KEY (salesperson_ID) REFERENCES EMPLOYEES(employee_ID)  
);
```

INSERT INTO to insert data to a table

```
-- syntax
INSERT INTO table_name (column1, column2, ...)
VALUES (value1, value2, ...);
```

```
-- example
-- 1. Insert a new book with only id and title
-- two single quotes (') to escape a single quote (') in a string
INSERT INTO books (id, title)
VALUES (1, 'Harry Potter and the Sorcerer's Stone');
-- 2. Insert a new book with all columns
INSERT INTO books
VALUES (1, 'Harry Potter and the Sorcerer's Stone', 'J.K. Rowling', 1997);
-- 3. Insert multiple books
INSERT INTO books (id, title, author, published_year)
VALUES
    (1, 'Harry Potter and the Sorcerer's Stone', 'J.K. Rowling', 1997),
    (2, ...),
    (3, ...);
```



Why would the database reject an **INSERT**?

An **INSERT** succeeds only if (1) it follows the **INSERT** syntax and (2) the values satisfy the table definition (data types and constraints).

```
CREATE TABLE books (  
    id int,  
    title text,  
    author text,  
    published_year int,  
    PRIMARY KEY (id),           -- id is the primary key  
    CHECK (published_year >= 1900) -- published_year must be >= 1900  
);
```

```
INSERT INTO books  
VALUES ('Harry Potter and the Sorcerer's Stone');
```

```
INSERT INTO books (id, title)  
VALUES ('Harry Potter and the Sorcerer's Stone');
```



Why would the database reject an **INSERT**?

An **INSERT** succeeds only if (1) it follows the **INSERT** syntax and (2) the values satisfy the table definition (data types and constraints).

```
-- continued from previous slide
```

```
INSERT INTO books (id)
VALUES (1, 'Harry Potter and the Sorcerer's Stone');
```

```
INSERT INTO books (id)
VALUES ('Harry Potter and the Sorcerer's Stone');
```

```
INSERT INTO books (title)
VALUES ('Harry Potter and the Sorcerer's Stone');
```

```
INSERT INTO books (id, title, author, published_year)
VALUES (1, 'Harry Potter and the Sorcerer's Stone', 'J.K. Rowling', 1897);
```

Why a **student_book** table?

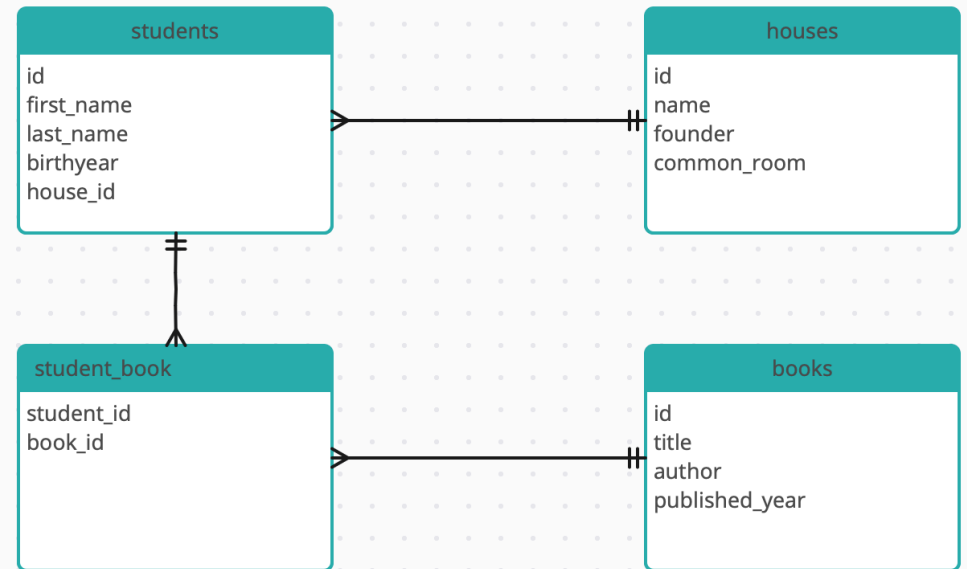
Relationship between **students** and **books**

- ➡ A student can appear in many books
- ➡ A book can feature many students
- ➡ **Many-to-many relationship**

student_book : **Associative Entity** to represent the many-to-many relationship without duplicating data

🤔 **Primary Key?**

🤔 **Foreign Key?**



student_book: composite PK + two FKs

```
CREATE TABLE student_book (  
    student_id int,  
    book_id int,  
    -- Composite PK: student_id and book_id together form a PK  
    PRIMARY KEY (student_id, book_id),  
    -- FK: student_id to reference the students table  
    FOREIGN KEY (student_id) REFERENCES students(id),  
    -- FK: book_id to reference the books table  
    FOREIGN KEY (book_id) REFERENCES books(id)  
);
```

Insert data into books table

With column names

```
-- Insert data into specific columns in the books table
INSERT INTO books (id, title, author, published_year) VALUES
-- each entry is separated by a comma ,
(201, 'Harry Potter and the Philosopher's Stone', 'J.K. Rowling', 1997),
(202, 'Harry Potter and the Chamber of Secrets', 'J.K. Rowling', 1998),
(203, 'Harry Potter and the Prisoner of Azkaban', 'J.K. Rowling', 1999),
(204, 'Harry Potter and the Goblet of Fire', 'J.K. Rowling', 2000),
(205, 'Harry Potter and the Order of the Phoenix', 'J.K. Rowling', 2003),
(206, 'Harry Potter and the Half-Blood Prince', 'J.K. Rowling', 2005),
-- last entry without a comma
(207, 'Harry Potter and the Deathly Hallows', 'J.K. Rowling', 2007);
```

Insert data into student_book table

Without column names

```
INSERT INTO student_book VALUES  
(101, 201), (101, 202), (101, 203), (102, 201), (102, 203),  
(102, 204), (103, 201), (103, 204), (104, 202), (104, 205),  
(105, 203), (105, 206), (106, 201), (106, 203), (106, 207),  
(107, 205), (107, 206), (108, 204), (108, 206), (109, 203),  
(109, 207), (110, 205), (111, 204), (112, 201), (113, 203),  
(114, 202), (115, 205), (116, 204), (117, 201), (118, 202);
```



Add `movies` table to Harry Potter database

- Table name: `movies`
- Columns:
 - `id` int
 - `main_char_id` int
 - `title` text
 - `director` text
 - `year` int
- Constraints:
 - primary key: `id`
 - foreign key: `main_char_id` → `id` in `students` table



Insert data into **movies** table

```
1, 101, 'Harry Potter and the Sorcerer''s Stone', 'Chris Columbus', 2001
2, 101, 'Harry Potter and the Chamber of Secrets', 'Chris Columbus', 2002
3, 101, 'Harry Potter and the Prisoner of Azkaban', 'Alfonso Cuarón', 2004
4, 101, 'Harry Potter and the Goblet of Fire', 'Mike Newell', 2005
5, 101, 'Harry Potter and the Order of the Phoenix', 'David Yates', 2007
6, 101, 'Harry Potter and the Half-Blood Prince', 'David Yates', 2009
7, 101, 'Harry Potter and the Deathly Hallows – Part 1', 'David Yates', 2010
8, 101, 'Harry Potter and the Deathly Hallows – Part 2', 'David Yates', 2011
```

FK constraint violation example

```
INSERT INTO movies (id, main_char_id, title, director, year)
VALUES (9, 999, 'Harry Potter and the Cursed Child', 'John Doe', 2016);
```

Rejected: FK constraint violation (main_char_id 999 doesn't exist in `students` table)

The rest of DDL and DML

DDL

- `CREATE TABLE` : ~~create a new table~~
- `ALTER TABLE` : modify a table
- `DROP TABLE` : remove a table

DML

- `INSERT INTO` : ~~insert data into a table~~
- `UPDATE` : modify existing data
- `DELETE` : remove existing data

DDL: ALTER TABLE

Add columns

```
ALTER TABLE mytable  
ADD COLUMN column_name numeric;
```

Add constraints

```
ALTER TABLE mytable  
ADD CONSTRAINT constraint_name  
FOREIGN KEY (column_name) REFERENCES other_table(other_column);
```

DDL: ALTER TABLE

Remove columns

```
ALTER TABLE mytable  
DROP COLUMN column_name;
```

Rename the table

```
ALTER TABLE mytable  
RENAME TO new_table_name;
```

DDL: DROP TABLE

DROP TABLE removes a table and all its data

```
CREATE TABLE mytable (id int);    -- mytable is created
DROP TABLE mytable;              -- Drop mytable
DROP TABLE mytable;              -- Error because mytable doesn't exist anymore
```

DROP TABLE IF EXISTS removes a table only if it exists

```
CREATE TABLE mytable (id int);    -- mytable is created
DROP TABLE mytable;              -- Drop mytable
DROP TABLE IF EXISTS mytable;     -- No error even if mytable doesn't exist
```

DML: UPDATE table

```
UPDATE table_name  
SET column1 = value1, column2 = value2, ...  
WHERE condition;
```

```
UPDATE books  
SET published_year = 1998  
WHERE id = 1;
```

DML: DELETE FROM table

```
DELETE FROM table_name  
WHERE condition;
```

```
DELETE FROM books  
WHERE id = 1;
```



Update and delete data from the Movies table

1. Remove director column from the movies table
2. Add a new column `director_id` to the movies table
3. Delete the movie with id 1 from the movies table
4. Update the year of the movie with id 2 to 2003

Wrap Up

DDL: Data Definition Language

- `CREATE TABLE` : create a new table
- `ALTER TABLE` : modify a table
- `DROP TABLE` : remove a table

DML: Data Manipulation Language

- `INSERT INTO` : insert data into a table
- `UPDATE` : update data in a table
- `DELETE FROM` : remove data from a table